

BitAI: The AI Layer for Blockchains secured by Bitcoin Tapscripts

Luke Pacman

luke@bitai.foundation

Issac Simpson

issac@bitai.foundation

Abstract

BitAI is a novel protocol that enables the execution of AI models in Bitcoin. At its core, it employs optimistic computation, operating on the assumption that agents act honestly unless proven otherwise by challengers using mathematical proofs. By utilizing Bitcoin on-chain transactions, BitAI ensures the correctness of AI computations while inheriting the security of Bitcoin consensus. This protocol unlocks the potential for a new class of AI-powered applications, offering sovereign security in a fully trustless and autonomous manner.

1 Introduction

The rapid proliferation of AI technologies has driven significant advancements across various industries. However, the increasing reliance on centralized architectures introduces systemic risks, including opaque decision-making processes, data monopolies, and single points of failure. Decentralized AI aims to address these challenges, but existing approaches often fall short due to their reliance on questionable assumptions.

A major challenge in decentralized AI is verification. Current decentralized AI protocols [6, 7] typically rely on validator redundancy, requiring multiple nodes to redundantly verify AI computations. This method is not only resource-intensive but also exposes the system to additional attack vectors. Another issue is determinism, as many protocols [7] demand identical hardware specifications to ensure consistent results across participating nodes. These requirements are impractical in a truly decentralized and hardware-diverse environment.

Applications requiring sovereign security, such as DeFAI, cannot depend on these incomplete solutions. Sovereign security necessitates a system that is not only trustless but also deterministic and verifiable without relying on weak assumptions.

BitAI addresses these challenges by introducing a Decentralized AI Protocol built on Bitcoin Tapscripts. By leveraging Bitcoin Tapscripts for verification, BitAI eliminates the need for validator redundancy, securing verification through mathematical proofs. This approach significantly reduces resource overhead while enhancing security. Moreover, the protocol employs integer-only quantization for AI model weights and inference, ensuring hardware-agnostic determinism that is fully compatible with Bitcoin Script’s integer arithmetic constraints. These innovations enable a decentralized AI protocol that inherits the robust security of Bitcoin consensus.

This whitepaper outlines the design and implementation of BitAI, presenting it as a groundbreaking protocol for AI-powered dapps that demand the highest levels of security and trustlessness.

Organization. We begin by introducing the necessary background, building blocks, and the establishment of AI inference verification using Bitcoin Tapscripts in Section 2. Next, we present the BitAI architecture and its components in Section 3. In Section 4, we detail the implementation of the BitAI protocol for securely verifying AI inference on Bitcoin. Finally, we address how the protocol manages large AI models in Section 5.

2 Background and establishment of AI inference verification

Verification of an AI inference is processed through a series of structured steps. In this section, we present necessary background, building blocks and details of these steps to see how the verification is established in Bitcoin.

2.1 Integer-only Quantization

A typical AI model consists of three components: the architecture, weights, and inference function. The inference function performs feed-forward computations through the layers of the model, which are defined by the architecture and use floating-point precision weights.

In a distributed and decentralized system, determinism is a prerequisite for one machine to verify the inference results of another. To achieve this, we need to perform quantization, which ensures that all computations during the feed-forward process are carried out using only integer numbers.

Additionally, integer-only quantization enables the computations to be executed using Bitcoin Scripts, which only support integer arithmetic operations. This capability is essential because we aim to utilize Bitcoin Tapscripts for inference verification, a process that will be explained shortly.

Depending on the characteristics of each model type, different integer-only quantization techniques may be applied. For smaller models, common post-training quantization methods often suffice. However, for larger models (e.g., LLMs), specialized quantization approaches, such as I-LLM [4], are needed. These techniques are specifically designed to address the challenges of integer-only quantization in LLMs, including significant fluctuations in activations across channels and tokens in both linear and non-linear operations, maintaining accuracy comparable to the original floating-point precision models.

2.2 Inference function's instructions compilation

After quantization, the inference function is broken down into multiple instructions. This step produces an intermediate representation of the original function, where each instruction is a simple arithmetic operation, making it straightforward to implement later in Bitcoin Script.

This approach is based on a key observation: the majority of computations in an AI model's inference function involve matrix multiplication, which can be decomposed into basic arithmetic operations such as addition, subtraction, multiplication, and division. Furthermore, other complex operations can often be approximated and re-expressed using these fundamental operations.

For example, consider an inference function that performs a simple matrix multiplication as follows:

```
input_vector = np.array(input_vector)
weight_matrix = np.array(weight_matrix)
output = np.dot(input_vector, weight_matrix)
```

if input_vector is a 1 x 3 vector: [i1, i2, i3]

and weight_matrix is a 3 x 2 matrix: [[w1, w2], [w3, w4], [w5, w6]]

then output is a 1 x 2 vector: [o1, o2]

The matrix multiplication can be compiled into the following instructions:

```
i1 * w1 = v1
i2 * w3 = v2
i3 * w5 = v3
0 + v1 = v4
v4 + v2 = v5
```

```
v5 + v3 = o1  
  
i1 * w2 = v7  
i2 * w4 = v8  
i3 * w6 = v9  
0 + v7 = v10  
v10 + v8 = v11  
v11 + v9 = o2
```

In the above instructions, *i*, *w*, *v*, and *o* represent "variable identifiers" rather than actual values, similar to how a compiler in a high-level programming language translates functions into an intermediate representation. These identifiers symbolically reference inputs, weights, intermediate results, and outputs, respectively. They are not tied to specific numerical values but instead serve as a way to structure the computation in a step-by-step manner, making it easier to interpret and execute the instructions in the target environment, such as Bitcoin Script.

2.3 Inference Verification with Bitcoin Taproot

The core idea of BitAI is to enable the computations of an inference function to occur optimistically. The agent providing inference outputs is assumed to be honest unless a challenger proves otherwise through a challenge-response game conducted within a predefined challenge period. This mechanism ensures the verification of AI inferences in BitAI inherits the security of Bitcoin consensus.

In this section, we provide the necessary background and building blocks before presenting the Taproot trees employed in the protocol.

2.3.1 Lamport Signatures

Lamport signatures are secure one-time signatures. The scheme assumes the message being signed is 256 bits long. This is ensured by hashing any message of arbitrary length using the SHA-256 hash function. The public key in this scheme consists of 256 pairs of hashes, totaling 512 hashes. To sign a message, the user reveals one preimage from each pair, selecting it based on the corresponding bit of the message. The verifier re-hashes the preimages and compares them against the message and public key to confirm their validity.

Bitcoin Script can verify Lamport signatures on values up to 33 bits by following this process:

1. For each preimage, compute its hash and compare it with two target hashes (representing a pair in the public key) embedded in the script.
2. If the hash matches the first value in the pair, it represents a 0-bit, and the script proceeds without action.
3. If it matches the second value, it represents a 1-bit, and the script adds the corresponding power of 2 to an accumulator.

4. If the hash matches neither value, the signature is invalid, and the script halts.

The final value in the accumulator represents the signed number, constructed by summing powers of 2 for each 1-bit in its binary representation.

For example, a script to sign a 4-bit message M can be written as follows:

```
0
SWAP sha256 DUP <H(B_0_1)> EQUAL IF DROP <1> ADD ELSE <H(B_0_0)> EQUALVERIFY ENDIF
SWAP sha256 DUP <H(B_1_1)> EQUAL IF DROP <1<<1> ADD ELSE <H(B_1_0)> EQUALVERIFY ENDIF
SWAP sha256 DUP <H(B_2_1)> EQUAL IF DROP <1<<2> ADD ELSE <H(B_2_0)> EQUALVERIFY ENDIF
SWAP sha256 DUP <H(B_3_1)> EQUAL IF DROP <1<<3> ADD ELSE <H(B_3_0)> EQUALVERIFY ENDIF
```

To sign the value 6, the owner of B expands it to binary 0110, then pushes the corresponding preimages onto the stack.

2.3.2 Stateful Bitcoin Scripting

Even though Bitcoin's scripting language is stateless, one-time signature schemes like Lamport signatures make it possible to effectively preserve state across multiple Bitcoin transactions.

As demonstrated in Section 2.3.1, the script for verifying Lamport signatures tied to a public key enables a signer to set the value of a variable from one UTXO to another, thereby simulating state preservation in Bitcoin. Specifically, by extracting the message value and its corresponding (Lamport) commitment from the unlocking script of one UTXO, these values can then be passed forward to another UTXO.

2.3.3 Connectors

Connectors are a special output type used to verify the existence of a specific transaction ID in Bitcoin Script. To achieve this, we simply attach an output from that transaction into our spending transaction and check a pre-signed signature against all prevouts of the spending transaction. In Section 2.3.5, we will demonstrate how connector outputs are utilized in the BitAI protocol to enforce the order of transactions.

2.3.4 Covenant Simulation

Bitcoin covenants are mechanisms that enable a transaction's locking script to impose restrictions on how the coins in a UTXO can be spent in the future. This allows the execution of a state machine through a series of transactions, enhancing expressiveness for smart contract capabilities in Bitcoin.

Currently, such covenants are not feasible in Bitcoin without changes to its consensus. In the absence of these changes, a common approach to simulate covenants involves using pre-signed transaction signatures. These pre-signed transactions, collaboratively agreed upon by multiple

participants in a system or protocol, enforce specific predefined constraints, effectively mimicking the functionality of covenants.

2.3.5 Bitcoin Transactions Setup

We utilize the building blocks mentioned above to set up the necessary transactions for the challenger to disprove an incorrect claim by the agent.

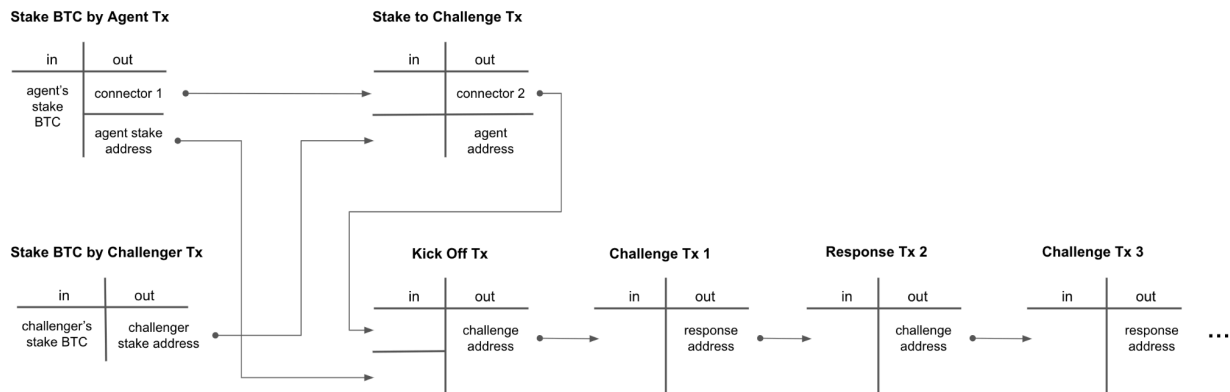


Fig. 1. Bitcoin transactions for the challenger to challenge and disprove an agent's incorrect claim

Stake BTC by Agent Transaction

In this transaction, the agent stakes by sending BTC to the following two outputs:

- Output 0: A connector that sends a small BTC amount (546 sats) to the agent's address.
- Output 1: Sends the staked BTC to the agent's stake address, a Taproot address committed by the following Tapscript:

```
<lock_time1> OP_CHECKSEQUENCEVERIFY OP_DROP <agent_public_key> OP_CHECKSIG
<setup_id> OP_EQUALVERIFY <agent_public_key> OP_CHECKSIGVERIFY <challenger_public_key>
OP_CHECKSIG
```

The Tapscript has two spending paths: either the agent can spend the funds after the lock time expires, or the agent and challenger can jointly cosign to spend the funds.

Stake BTC by Challenger Transaction

In this transaction, the challenger stakes by sending BTC to one output: the challenger's stake address, a Taproot address committed by the following Tapscript:

```
<lock_time2> OP_CHECKSEQUENCEVERIFY OP_DROP <challenger_public_key> OP_CHECKSIG
<setup_id> OP_EQUALVERIFY <challenger_public_key> OP_CHECKSIGVERIFY <agent_public_key>
OP_CHECKSIG
```

The Tapscript also has two spending paths: either the challenger can spend the funds after the lock time expires, or the agent and challenger can jointly cosign to spend the funds.

Stake to Challenge Transaction

If the challenger decides to challenge the agent's claim, they broadcast the stake-to-challenge transaction to Bitcoin.

This transaction has two inputs:

- Input 0: Connector output from the stake-BTC-by-agent transaction, ensuring the stake-to-challenge transaction is only valid if the stake-BTC-by-agent transaction is included in the Bitcoin chain.
- Input 1: Challenger's staked BTC output from the stake-BTC-by-challenger transaction.

The funds from these inputs are pre-agreed during the setup process to be sent to the following two outputs, where the unlocking script for input 0 requires the agent's presigned signature, and the unlocking script for input 1 requires cosigned signatures from both the agent and the challenger:

- Output 0: a connector sending a small BTC amount (546 sats) to the agent's address.
- Output 1: sends the challenger's staked BTC to the agent's address as collateral.

Kick-off Transaction

Next, the challenger broadcasts the kick-off transaction to Bitcoin. This transaction has two inputs:

- Input 0: connector output from the stake-to-challenge transaction, ensuring the kick-off transaction is only valid if the stake-to-challenge transaction is included in the Bitcoin chain.
- Input 1: agent's staked BTC output from the stake-BTC-by-agent transaction.

The funds from these inputs are pre-agreed during the setup process to be sent to the following output, where the unlocking script for input 0 requires the agent's presigned signature, and the unlocking script for input 1 requires cosigned signatures from both the agent and the challenger:

- Output 0: sends the agent's staked BTC to the challenge address to kick off the challenge-response procedure. The challenge address setup will be explained in the next section.

The two building blocks, the connector and covenant simulation using presigned transactions, are used in both the stake-to-challenge and kick-off transactions. In subsequent transactions, we will

demonstrate how Lamport signatures and stateful Bitcoin scripting are used for challenge and response transactions.

Challenge, Response, and Disprove Transactions

Typically, the agent's task is to provide inference outputs for a given inference request. However, there must also be a mechanism for the challenger to challenge and slash the agent's staked BTC on-chain if the agent provides an incorrect output.

BitAI implements this capability by creating the following three Taproot Trees [11] (or Taptrees) derived from the compiled inference function (i.e., instructions in Section 2.2):

1. Challenge Taptree
2. Response Taptree
3. Disprove Taptree

Each Tapleaf in the challenge tree contains a hashlock of a variable identifier from the instructions. The entire challenge tree represents all variable identifiers from the instructions. The preimages for these hashlocks are generated and kept secret by the challenger. Additionally, there is a timeout Tapleaf allowing the agent to send funds after the timeout.

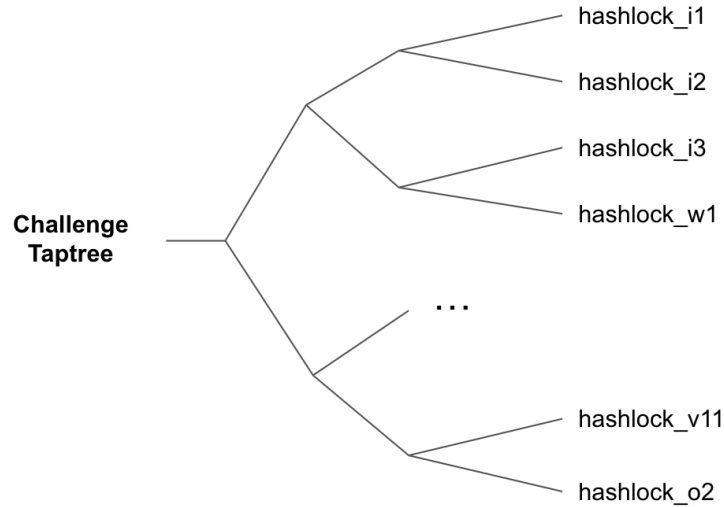


Fig. 2. Illustration of the Challenge Taptree for the matrix multiplication example from Section 2.2

Each Tapleaf in the response tree contains a hashlock and Lamport commitments for a variable identifier from the instructions. The entire response tree represents all variable identifiers from the instructions. The preimages for the Lamport commitments are generated and kept secret by the agent. Additionally, there is a timeout Tapleaf allowing the challenger to send funds after the timeout.

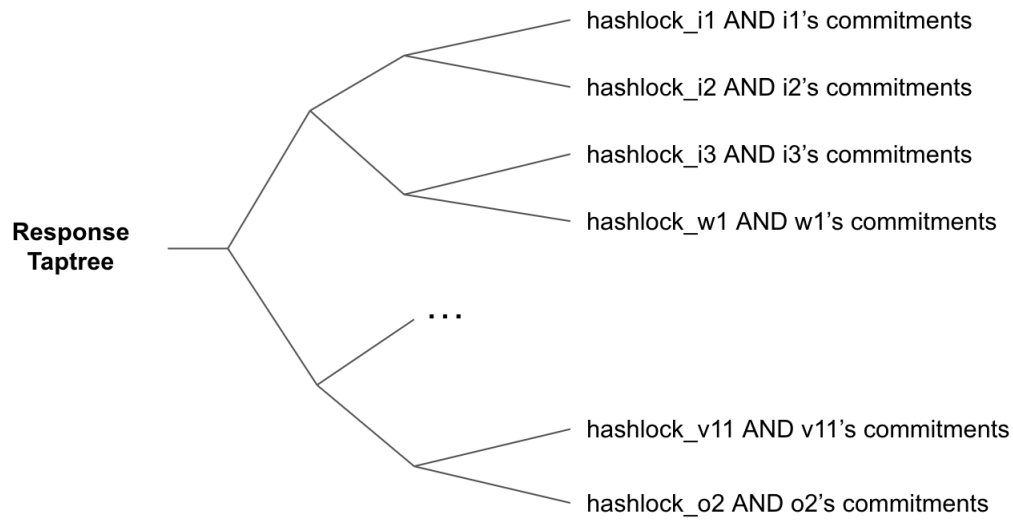


Fig. 3. Illustration of the Response Taptree for the matrix multiplication example from Section 2.2

Each Tapleaf in the disprove tree contains a Bitcoin Script implementing arithmetic calculations for an instruction (e.g., $a + b = c$, where a , b , and c are represented by Lamport commitments). The entire disprove tree implements all instructions from the inference function.

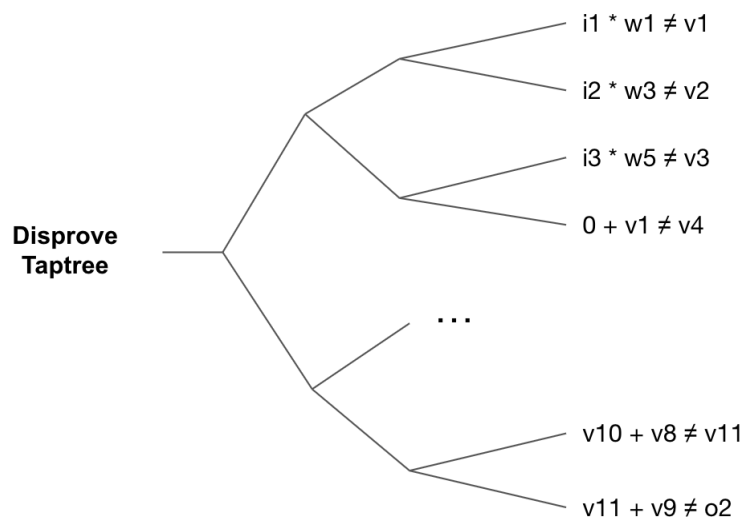


Fig. 4. Illustration of the Disprove Taptree for the matrix multiplication example from Section 2.2

The challenge address is committed from a large Taptree that combines the challenge Taptree and the disprove Taptree. The response address is committed from the response Taptree.

A sequence of transactions is pre-signed by the challenger and agent to perform multiple rounds of challenge-response:

1. The challenger chooses a challenge by revealing a preimage for one of the hashlocks in the challenge Taptree to send funds to the response address.
2. Due to the timeout, the agent is forced to reveal preimages for Lamport commitments (i.e., actual value) of the variable identifier corresponding to the revealed hashlock. This action sends funds back to the challenge address.
3. By applying a binary search, the challenger can quickly identify the agent's error after a few rounds of challenge-response, disprove the incorrect committed instruction in the disprove Taptree, and send funds to their address.

3 Architecture Overview

In this section, we present an overview of the BitAI architecture, describing all the actors participating in the system along with their responsibilities.

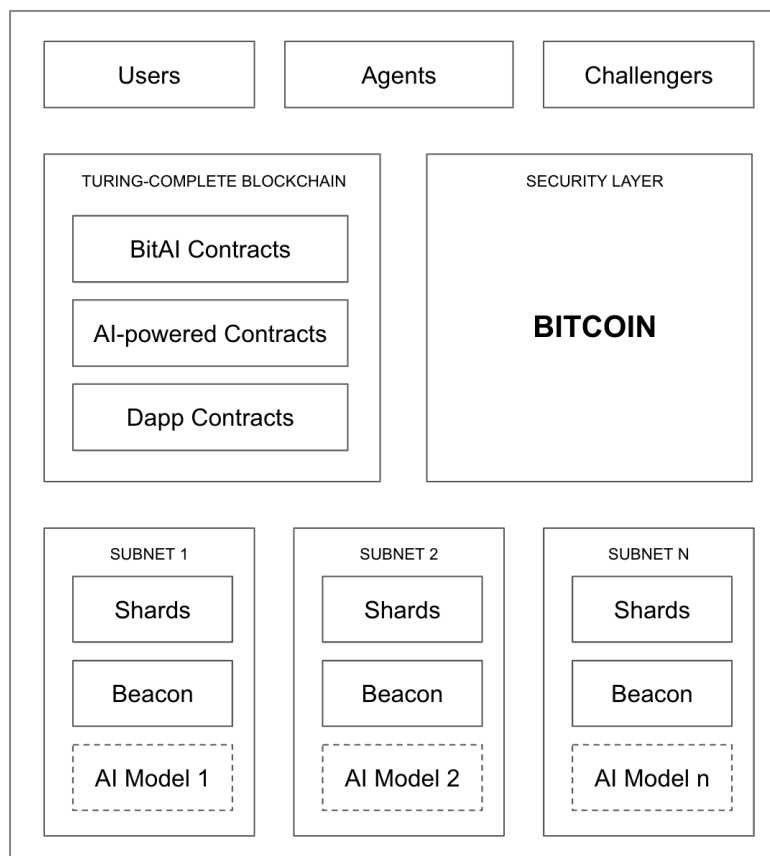


Fig. 5. BitAI Architecture and its components

3.1 Actors

User

Requests an inference and receives output from agent(s) for a particular AI model.

Agent

Responds to an inference requested by the user.

Challenger

Verifies the agent's output and slashes the agent's staked BTC by forcing it to participate in a challenge-response game if the output is not genuinely produced by the requested AI model.

Validator

There are two types of validators: shard validators and beacon validators. These validators collaboratively operate in a sharding network, allowing multiple shards to run simultaneously. This architecture improves latency and scalability through parallel processing.

- **Shard Validator:** Ensures that the setup process between the agent and challenger is conducted genuinely, as defined in the protocol.
- **Beacon Validator:** Validates output format for inferences provided by the agent to ensure that the challenger has sufficient data to execute slashing through the challenge-response mechanism.

BitAI Contracts

There are three main activities in the system:

1. Commitment and Pre-sign Setup
2. Inference
3. Slashing with the Challenge-Response mechanism

The BitAI contracts are deployed on a Turing-complete blockchain, such as Ethereum, to implement the logic and manage the state for commitment and pre-sign setup, inference, and staking. Slashing with the challenge-response mechanism is implemented using Bitcoin Tapscripts.

Consensus Layer

Defines Practical Byzantine Fault Tolerance (pBFT) consensus rules for setups and inferences to ensure data integrity for resolving future disputes.

Coordination Layer

Coordinates all components of the system through event emissions:

- Commitment and Pre-sign Setup (Section 4.1, 4.2, 4.3 and 4.4): Involves agents, shard validators, and challengers.
- Inference Requests and Responses (Section 4.5): Involves users, agents, and beacon validators.

Staking Layer

Allows anyone to stake and become a validator for a subnet. Validators are randomly assigned to either a shard or beacon, ensuring a secured and balanced distribution.

AI-powered contracts

These contracts utilize inference outputs from the execution of AI models submitted by agents to implement AI-powered on-chain applications. For example, an AI-powered portfolio management contract can use the inference results of an algorithmic trading model to make decisions (buy, sell, or hold) on funds deposited by clients. The process is fully autonomous and trustless.

Dapp contracts

Thanks to the composability of smart contracts, AI-powered contracts can leverage the liquidity and network effects of popular dApp contracts like Uniswap, Aave, and others to unlock unprecedented innovative applications.

3.2 Subnet and Sharding

BitAI is a multi-model, decentralized AI protocol designed to support diverse AI models through dedicated sub-networks. Each subnet operates as a self-contained environment, tailored to the unique computational and data requirements of its associated AI model, and is powered by a separate set of contracts, agents, challengers and validators.

Each subnet is further divided into multiple shard networks and a beacon network . This approach draws inspiration from Ethereum's sharding architecture that enhances scalability by leveraging parallelism.

In BitAI, shards are responsible for ensuring that the setup process between the agent and the challenger is conducted genuinely (as outlined in Sections 4.2, 4.3, and 4.4). Shards are randomly assigned to handle setup processes simultaneously, which means that validators within each shard only need to store a subset of the data rather than the full dataset for all setups.

Meanwhile, the beacon is responsible for ensuring that the output format of inferences provided by the agent is valid (as described in Section 4.5). Since each inference request is randomly assigned by the Consensus contract to one or more relevant setups, beacon validators must store the complete dataset for all setups. This ensures they have sufficient data to validate the outputs submitted by agents.

4 Protocol Implementation

4.1 Matching

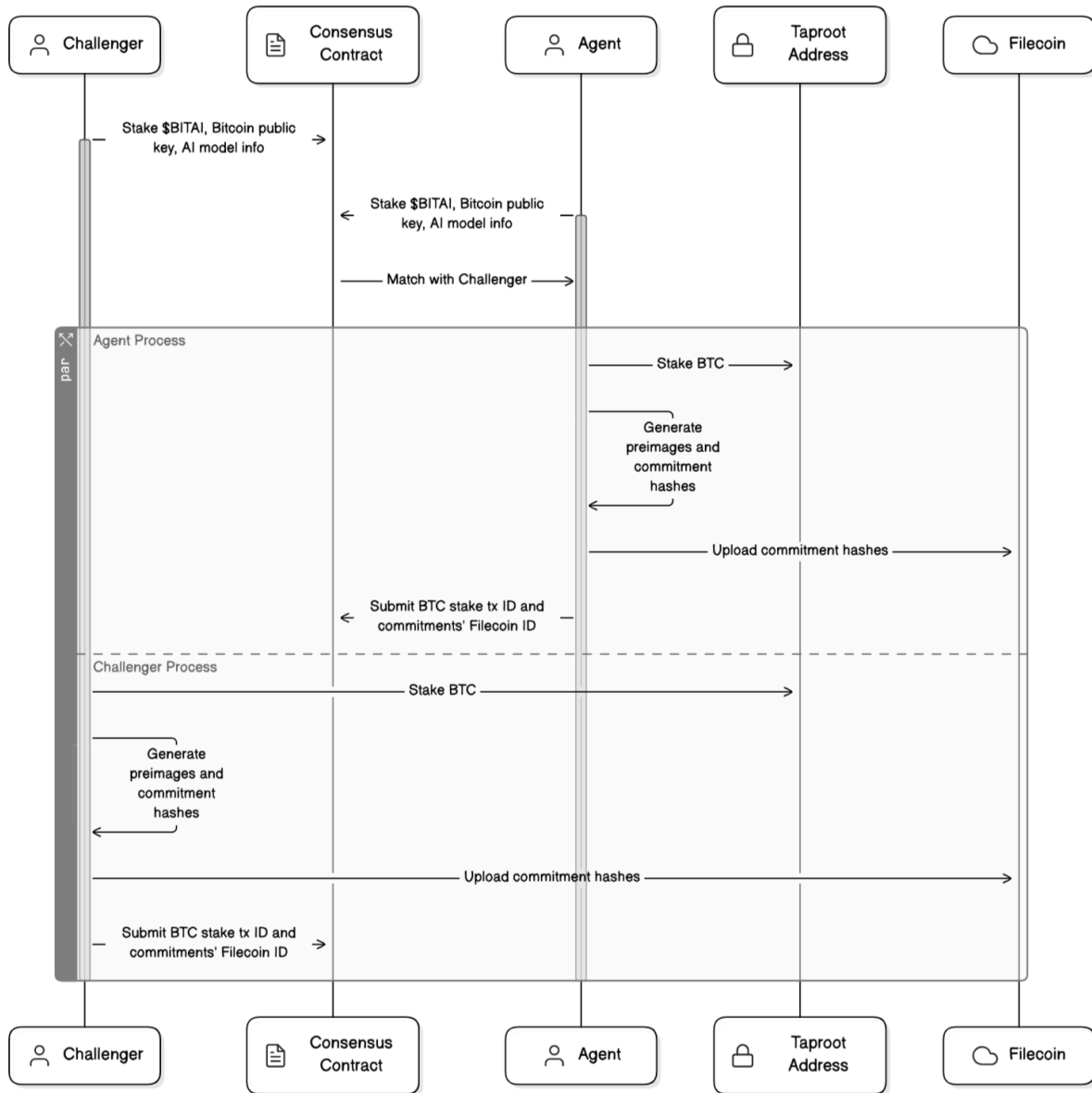


Fig. 6. The matching procedure between a challenger and an agent

1. Challenger stakes \$BITAI along with a Bitcoin public key and AI model information to the Consensus contract.
2. Agent stakes \$BITAI along with a Bitcoin public key and AI model information to the Consensus contract. It's randomly matched with a challenger by the contract.

3.1 Agent creates a stake-BTC-by-agent transaction (Section 2.3.5) and broadcasts it to the Bitcoin network.

3.2. Agent generates preimages and commitment hashes needed for a specific AI model which it's serving. The agent keeps the preimages secret and uploads the commitment hashes to a decentralized data storage like Filecoin; it has now a Filecoin ID for the upload.

3.3 Agent submits the stake transaction ID and the uploaded Filecoin ID to the contract.

4.1. Challenger creates a stake-BTC-by-challenger transaction (Section 2.3.5) and broadcasts it to the Bitcoin network.

4.2 Challenger generates preimages and commitment hashes needed for a specific AI model which it's serving. The challenger keeps the preimages secret and uploads the commitment hashes to a decentralized data storage like Filecoin; it has now a Filecoin ID for the upload.

4.3 Challenger submits the stake transaction ID and the uploaded Filecoin ID to the contract.

4.2 Challenge & Response Addresses Construction

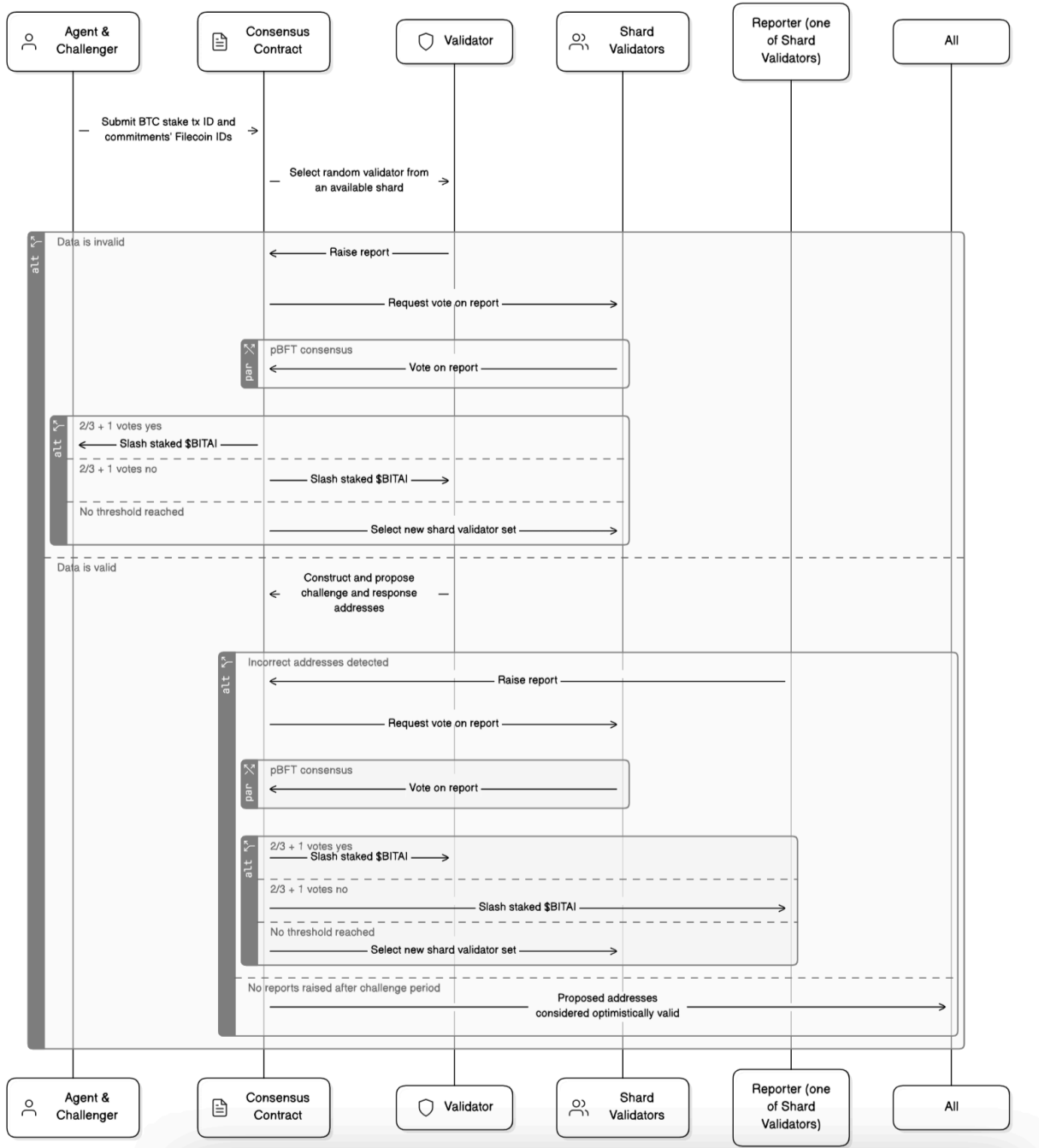


Fig. 7. Construction of Challenge & Response addresses by Shard validators

A random validator is selected from a random available shard to verify the stake transaction IDs and commitment hashes submitted by the agent and challenger. If it detects any submitted data as invalid, it raises a report by creating a transaction to the Consensus contract. Subsequently, all

validators of the shard are notified about the raised report and begin verifying the report themselves. They then vote on the report through a transaction to the contract. A practical Byzantine Fault Tolerance (pBFT) consensus with a $2/3 + 1$ threshold is implemented in the contract to decide whether the report is valid.

- If $2/3 + 1$ of the shard validator set votes "yes," the report is valid, and the bad actors who submitted incorrect data are slashed for their staked \$BITAI.
- If $2/3 + 1$ of the shard validator set votes "no," the report is invalid, and the proposed validator is slashed for their staked \$BITAI.
- If no threshold is reached, the contract randomly selects another shard validator set to restart the process.

If the selected validator does not detect any invalid submitted data, it proceeds to build challenge and response addresses from the submitted stake transaction IDs and commitment hashes, proposing these addresses through a transaction to the contract. These addresses are committed from Challenge, Response and Disprove Taptrees as mentioned in Section 2.3.5.

Other validators of the selected shard then verify the submitted addresses. If any of them detect incorrect addresses, they raise a report, triggering a pBFT consensus with a $2/3 + 1$ threshold within the contract to vote on the raised report.

- If $2/3 + 1$ of the shard validator set votes "yes," the report is valid, and the proposer is slashed for their staked \$BITAI.
- If $2/3 + 1$ of the shard validator set votes "no," the report is invalid, and the reporter is slashed for their staked \$BITAI.
- If no threshold is reached, the contract randomly selects another shard validator set to restart the process.

If, after the challenge period from the time the proposed addresses are submitted, no reports are raised, the proposal of the selected validator is considered optimistically valid.

4.3 Presigning By Challenger

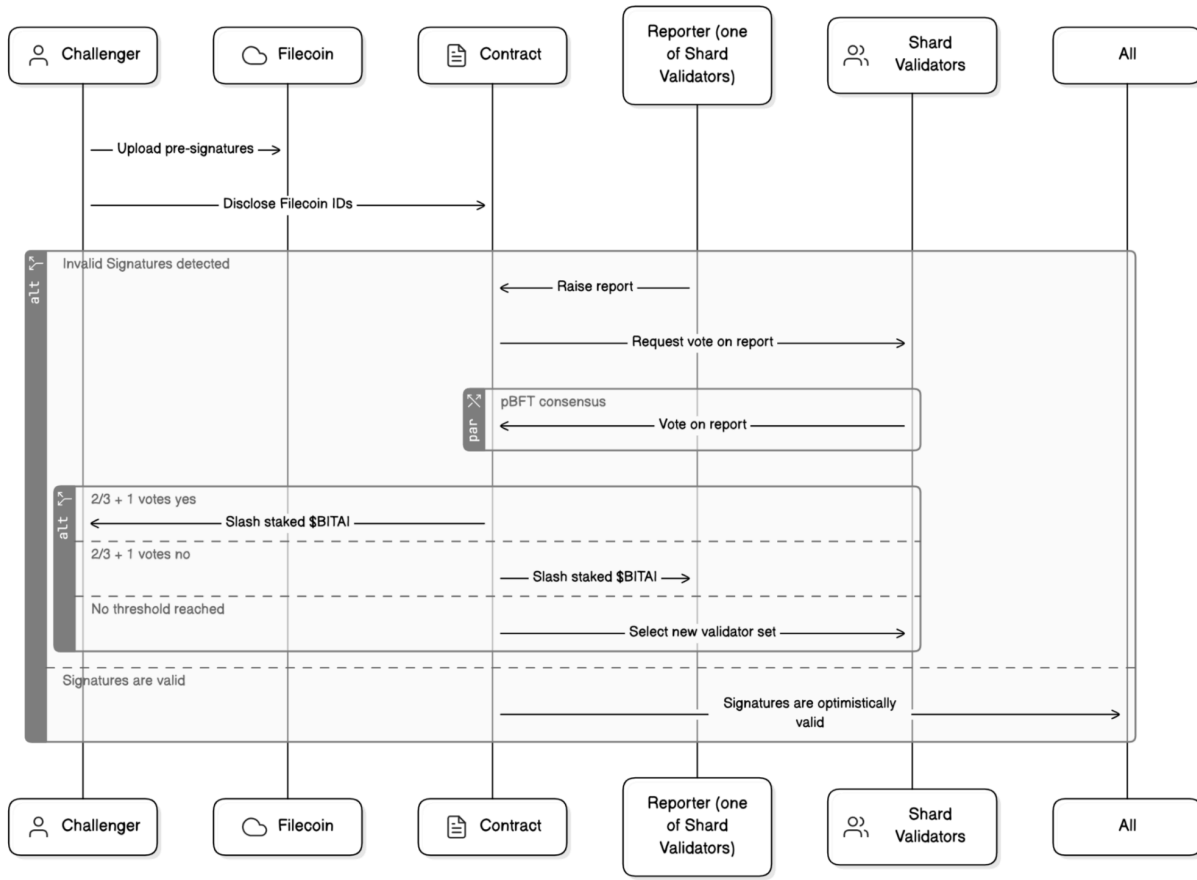


Fig. 8. The procedure for presigning simulated covenant transactions by challenger

1. The challenger proceeds to pre-sign all the necessary transactions built from the stake transaction IDs, commitment hashes, and challenge and response addresses submitted by the agent, shard validator, and itself in the previous steps. The pre-signatures are then uploaded to Filecoin, and the uploaded Filecoin IDs are disclosed to the contract through a transaction.

2. Each time the challenger's pre-signatures are submitted, all validators from a randomly selected available shard verify the signatures. If any validator detects invalid signatures or finds that the submitted signatures do not satisfy the predefined rules, they raise a report to the contract. A pBFT consensus with a $2/3 + 1$ threshold is then initiated to vote on the raised report.

- If $2/3 + 1$ of the shard validator set votes "yes," the report is valid, and the challenger is slashed for their staked \$BITAI.
- If $2/3 + 1$ of the shard validator set votes "no," the report is invalid, and the reporter is slashed for their staked \$BITAI.

- If no threshold is reached, the contract randomly selects another shard validator set to restart the process.

If, after the challenge period from the time the signatures are submitted by the challenger, no reports are raised, the signatures are considered optimistically valid.

4.4 Presigning By Agent

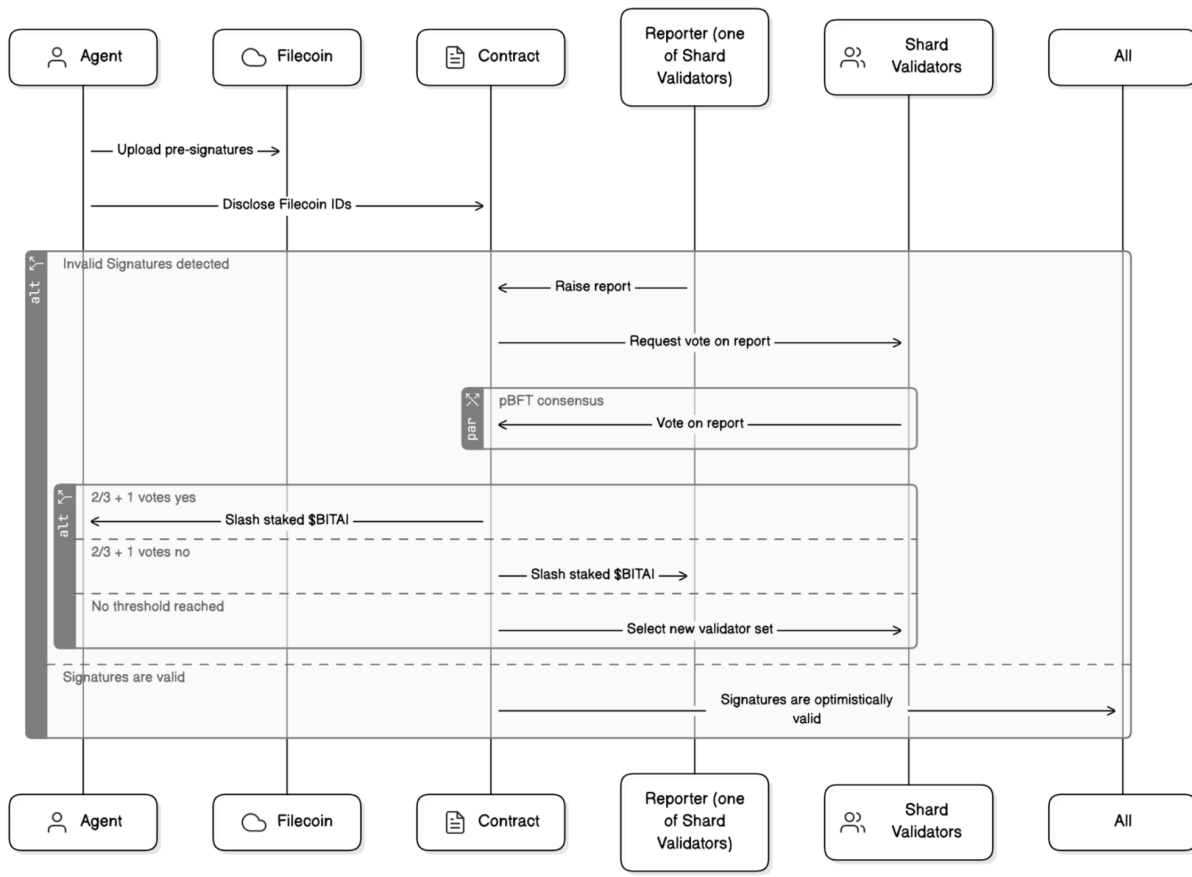


Fig. 9. The procedure for presigning simulated covenant transactions by agent

1. Next, it is the agent's turn to pre-sign all the necessary transactions, similar to what the challenger did in Step 6. The pre-signatures are then uploaded to Filecoin, and the uploaded Filecoin IDs are disclosed to the contract through a transaction.
2. The same verification process is carried out by the validators of a randomly selected available shard for the agent's pre-signatures, as was done for the challenger's pre-signatures.

4.5 Inference

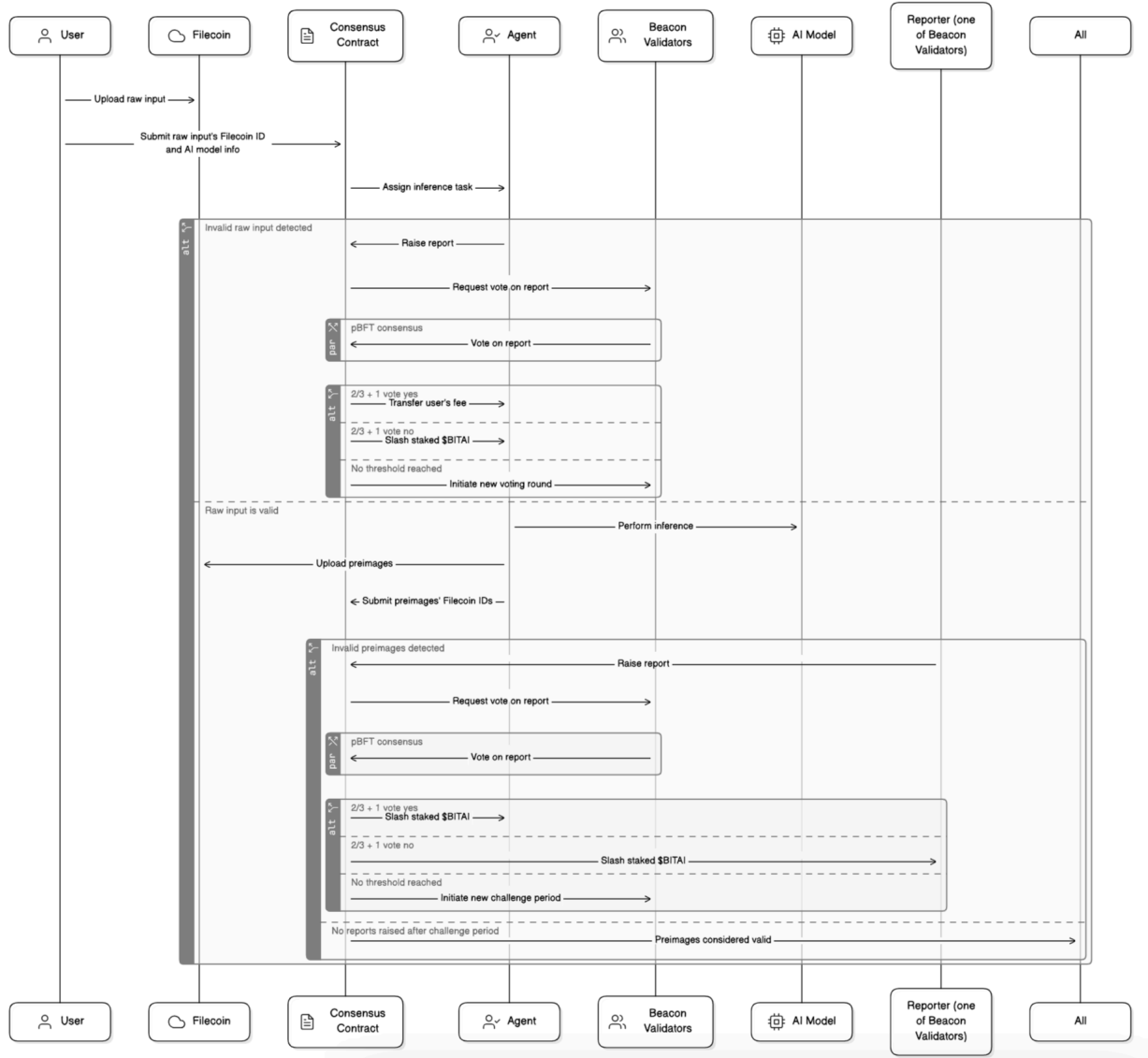


Fig. 10. The procedure for handling an inference in BitAI

1. The user uploads raw input (in numerical form) to Filecoin and submits the Filecoin upload ID, along with AI model information, to the Consensus contract.

2. The agent randomly assigned by the contract verifies the raw input. If it detects any invalid data, it raises a report by creating a transaction to the contract, triggering a pBFT consensus with a $2/3 + 1$ threshold to vote on the raised report.

- If $2/3 + 1$ of the Beacon validator set votes "yes," the report is valid, and the user's \$BITAI fee is transferred to the agent.

- If $2/3 + 1$ of the Beacon validator set votes "no," the report is invalid, and the agent is slashed for their staked \$BITAI.
- If no threshold is reached, the contract initiates a new voting round for the report.

3. If no issues are found, the agent performs AI model inference and then reveals the preimages of bit commitments for all numbers across the requested model layers, including the input layer. The revealed preimages are uploaded to Filecoin, and the upload IDs are submitted to the contract.

4. The Beacon validators verify the revealed preimages of the model layers. If any of them detect invalid preimages, they raise a report to the contract, triggering a pBFT consensus with a $2/3 + 1$ threshold to vote on the raised report.

- If $2/3 + 1$ of the Beacon validator set votes "yes," the report is valid, and the agent is slashed for their staked \$BITAI.
- If $2/3 + 1$ of the Beacon validator set votes "no," the report is invalid, and the reporter is slashed for their staked \$BITAI.
- If no threshold is reached, the contract initiates a new challenge period round for the revealed preimages.

If, after the challenge period from when the preimages are revealed by the agent, no reports are raised, the preimages are considered optimistically valid.

4.6 Disprove

If the challenger detects an incorrect output for an inference request provided by the agent, it can broadcast the pre-signed stake-to-challenge and kick-off transactions to the Bitcoin network to initiate a challenge-response game. Subsequently, the challenger and the agent broadcast a sequence of challenge-response transactions, enabling the challenger to identify an error in the agent's claims. If an error is proven, the challenger can slash the agent's staked BTC by broadcasting a disprove transaction.

The details of these transactions and how they enable disproving incorrect claims through the challenge-response mechanism are outlined in Section 2.3.5.

5 Large AI model with collaborative sub-agents

Many recent large AI models, such as large language models (LLMs) often have more than 100 billion parameters and using these models requires high-end hardware which poses a significant barrier for participants and limits decentralization. In this protocol, we propose a novel approach to address this challenge by collaboratively joining the resources of multiple parties.

The core idea of the approach is to split the parameters and computation of a large AI model into multiple smaller parts, each sub-agent is responsible for several consecutive layers (or Transformer blocks [8] in case of LLM model) so it only needs to hold a small fraction of the total parameter count. Before each inference session, the Consensus contract finds a chain of available sub-agents that collectively hold all model layers.

Once the sub-agent chain is formed, the selected sub-agents collaboratively perform inference steps, with the output of one sub-agent serving as the input for the next. This communication occurs through the Consensus contract until the user receives the final output.

If a sub-agent fails during inference (i.e., it does not provide the required output for its assigned inference step before the timeout), the Consensus contract will reassign the task to another available sub-agent by emitting an event and providing the necessary data to the replacement sub-agent.

6 Conclusion

BitAI represents a paradigm shift in decentralized AI by addressing the fundamental challenges that have long hindered its practical adoption. By leveraging Bitcoin Tapscripts for verification and introducing hardware-agnostic determinism through integer-only quantization, BitAI provides a secure, efficient, and scalable framework for decentralized AI applications.

This innovation unlocks new possibilities for applications demanding sovereign security, such as DeFAI, and sets a new standard for verifiability and trustlessness in the AI space. The advancements outlined in this whitepaper not only address existing shortcomings but also pave the way for broader adoption of decentralized AI in critical domains.

References

[1] Satoshi Nakamoto: Bitcoin: A Peer-to-Peer Electronic Cash System.

<https://bitcoin.org/bitcoin.pdf>

[2] Ethereum.org: Ethereum whitepaper, <https://ethereum.org/en/whitepaper/>

[3] Leslie Lamport. Constructing digital signatures from a one way function. Technical Report CSL-98, October 1979

- [4] Xing Hu, Yuan Cheng, Dawei Yang, Zhihang Yuan, Jiangyong Yu, Chen Xu and Sifan Zhou. I-LLM: Efficient Integer-Only Inference for Fully-Quantized Low-Bit Large Language Models. arXiv:2405.17849v2, 2024
- [5] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, Colin Raffel. PETALS: Collaborative Inference and Fine-tuning of Large Models. arXiv:2209.01188v2, 2023
- [6] Yuma Rao. Bittensor: A Peer-to-Peer Intelligence Market.
<https://bittensor.org/wp-content/uploads/2024/02/bittensor.org-whitepaper.pdf>
- [7] Sig Moid: Eternal AI: A Decentralized AI Network.
<https://eternalai.org/whitepaper.pdf>
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin. Attention Is All You Need. 2017
- [9] Salvatore Ingala. General smart contracts in bitcoin via covenants. <https://merkle.fun>
- [10] Jason Teutsch, Christian Reitwießner. A scalable verification solution for blockchains - TrueBit protocol. 2017
- [11] Pieter Wuille, Jonas Nick, Anthony Towns. Bip 341, Taproot: Segwit version 1 spending rules. 2020